

Orderless / Menu-Only Pod Support — Implementation Audit

Date: September 3, 2026

Status: Audit only. No product behavior was changed.

Scope: Schema, admin, vendor dashboard, pod dashboard, customer storefront, cart, checkout, payments, routing, menu, onboarding/readiness, claiming, analytics, notifications, APIs, copy, and edge cases.

1. Executive Summary

Open Order already separates **public visibility** from **orderability**, but it has **no first-class menu-only product state**. Every active vendor is treated as an ordering candidate that is either live, paused, closed, hidden, or “setup incomplete.”

The closest existing levers are:

Lever	What it actually means	Why it cannot be reused
<code>Vendor.mennyuOrdersPaused</code> / <code>Pod.mennyuOrdersPaused</code>	Temporary intake pause. Deliverect busy-mode writes the vendor flag. Copy is “paused / not accepting orders.”	This is an operational pause, not a durable product mode. Using it for menu-only would fight POS webhooks, hide the wrong UI, and look like an outage.
<code>Vendor.isActive</code> / <code>Pod.isActive</code>	Existence + public listing. Inactive pods 404. Inactive vendors are hidden.	Menu-only vendors and pods must remain publicly listed.
<code>PodVendor.isActive</code>	Hide this vendor from this pod’s public page.	Visibility, not commerce mode.
<code>orderRoutingMode</code> / <code>menuSource</code>	How paid orders are sent and which catalog is live.	Routing is not orderability. Overloading it would revive menu-source ownership bugs.
<code>MenuItem.isAvailable</code>	Item sold-out / available to sell.	Menu-only items must still look available for browsing.
Stripe / POS readiness	Required today before <code>canAcceptOrders</code> .	Menu-only vendors must not be forced through payout or routing setup.

Architectural change required: add explicit `orderingEnabled` flags on `Pod` and `Vendor` (default `true`), compute **effective orderability** in one helper, and thread that helper through storefront, cart, checkout, readiness, and dashboards.

Major risks if this is done poorly:

1. **Storefront already conflates “not orderable” with “unavailable.”**
`VendorMenuItemCard` treats `orderingDisabled` as item-unavailable, which greys out the menu and blocks item browsing. A menu-only vendor would look sold out unless this is split.
2. **Menu-source ownership.** Recent routing/menu-source work must not be touched when toggling orderability. Disabling ordering must not change `menuSource`, `orderRoutingMode`, published versions, or `isAvailable`.
3. **Pause vs menu-only copy.** Customers, admins, and pod owners currently see missing Stripe as “not accepting orders” / “needs payment.” Intentional menu-only must not look like a broken launch.
4. **Migration default.** Existing production vendors/pods must default to **orderable**. A `false` default would silently disable commerce.
5. **No unclaimed-vendor create path.** Public listing does not require an owner, but the only in-app `vendor.create` always creates a `VendorMembership` owner. Concierge “staff creates vendor + menu, then vendor claims later” needs new admin create + claim work (Phase H). It is not required for the first menu-only slice.

Good news: the current stack is closer than it looks.

- Public appearance already does **not** require Stripe or POS (`isVendorPublicProfileReady` vs `getVendorOperationalMissingItems`).
- `addCartItem` and `validateCartForOrder` already hard-reject non-orderable vendors via `getVendorOrderabilityInPod`.
- Admin Vendor Detail and Admin Pod Detail already have **Ordering controls**, but they only pause (`mennyuOrdersPaused`).
- Menu Builder already works for `menuSource = open_order` / `orderRoutingMode = manual_dashboard` without taking an order.

The work is to introduce a durable **intent layer** above pause/hours/Stripe/routing, then stop treating “cannot take a paid order right now” as a single customer state.

2. Current Architecture Findings

2.1 How orderability works today

There is a real two-layer model in `src/lib/vendor-readiness-states.ts` :

1. **Public profile readiness** — name, description, banner, cuisine, published/operational menu, customer hours. Controls whether the vendor appears on the pod page. Missing any of these → `hidden` (public 404 / omitted from grid).
2. **Operational readiness** — Stripe charges+payouts, routing/POS ready, Deliverect/Square mapping, at least one available item, pod active, membership active, not paused, currently open. Controls whether customers can order.

`getVendorOrderabilityInPod ( src/lib/vendor-orderability-in-pod.ts )` is the cart/checkout gate. When a readiness bundle is supplied, it uses the full operational check. When it is omitted, it **fail-opens** past Stripe/POS and only checks pause/active/hours.

Customer-facing derived states today:

Internal	Customer label	Typical cause
<code>hidden</code>	Vendor omitted / 404	Incomplete public profile, inactive, not in pod
<code>live</code>	"Open for orders"	Operationally ready and open
<code>visible_not_accepting</code>	"Not accepting orders right now" or "Closed right now"	Missing Stripe/POS, paused, outside hours, pod paused

There is **no** "menu only by design" state. A vendor with a great public menu and no Stripe is already visible, but the UI treats that as incomplete setup.

2.2 Implicit "every vendor should become orderable"

These are the strongest implicit requirements:

- `VENDOR_SETUP_REQUIRED_CHECKLIST_KEYS` always includes `stripe` , `pos` , `menu_available` , `pod_invite` .
- `POD_SETUP_REQUIRED_CHECKLIST_KEYS` includes `vendor_ready` = "at least one orderable vendor."
- `derivePodAttentionItems` warns when `orderableVendorCount === 0` .
- Admin pod overall status can become `no_orderable_vendors` .
- Admin vendor overall status defaults toward "Accepting orders" / "Setup required."
- Vendor nav always includes Orders, Kitchen, Payouts.
- Vendor dashboard copy: "finish the checklist so customers can order without surprises."
- Pod dashboard sidebar: "N orderable vendors."
- Customer pod ticker: "Open for orders" / "Not accepting orders."
- Vendor cards CTA: "Order now" vs "View menu" only when `availability.unavailable` .

2.3 Pause is already a different concept

`mennyuOrdersPaused` is wired through:

- Admin pause/unpause actions (`adminPauseVendorOrdering` , `adminPausePodOrdering`)
- Vendor kitchen/dashboard pause toggles
- Deliverect busy-mode webhook (`src/services/deliverect-busy-mode-webhook.service.ts`)
- Entity deletion (soft-delete also pauses)
- Cart/checkout error `VENDOR_PAUSED_MENNYU` / `POD_ORDERS_PAUSED`

This must remain a **temporary** “stop new tickets” switch. Menu-only is a **product configuration** that should survive hours, POS busy mode, and later re-enablement.

2.4 Public pods are already ungated

`Pod.isActive` defaults to `true` . The public pod page only requires an active pod. A pod with zero Stripe accounts and zero routing can already be public **if** attached vendors pass public-profile checks.

So “menu-only public pod” is not blocked by a launch gate in the customer app. It is blocked by **UX and readiness semantics**: dashboards scream, cards say “not accepting,” item cards look unavailable, and Stripe/POS remain required for “healthy.”

2.5 Cart/checkout protection is uneven

Path	Full readiness (Stripe/POS/mapping)?	Notes
<code>addCartItem</code>	Yes	Loads readiness with Deliverect mapping integrity
<code>validateCartForOrder</code> / <code>createOrderFromCart</code>	Yes	Same
<code>validateCartItemsForDisplay</code>	No	Hours/pause/pod-vendor only after item availability
<code>updateCartItem</code>	No	Item availability + modifiers; does not re-check vendor orderability
<code>createPaymentIntent</code>	No	Assumes order already created; does not re-check vendor Connect

UI hiding is therefore not enough, but checkout is already a hard gate for missing Stripe/POS. Menu-only must plug into the same gate with a **new reason code**, and display/update paths must catch up.

2.6 Vendor claiming / unclaimed public vendors

- `Vendor` has no `ownerId`. Ownership is `VendorMembership`.
- Public visibility does **not** require a membership.
- Admin vendor list already shows `"—"` when there are no owners.
- The only in-app `prisma.vendor.create` is `createVendorProfile`, which **always** creates an owner membership.
- `PodVendorInvite` can target an **existing** vendor (`targetVendorId`) or invite someone who then creates a vendor account.
- There is no claim token for "this pre-built unclaimed vendor is yours."
- Admin API `POST /api/admin/vendor-users` can attach a user to an existing vendor (secret/bootstrap, not a customer claim flow).

Flag: staff cannot currently create an unclaimed vendor in the product UI. Schema allows it; product does not. This is Phase H, not a blocker for menu-only browsing of vendors that already exist.

3. Proposed State Model

3.1 New persisted fields

Add two booleans. Do **not** add an enum of mixed runtime states. Do **not** name the stored field `isOrderable` — that should stay derived.

```
model Pod {
  /// Durable product mode. When false, customers cannot place orders at any
  vendor
  /// in this pod. Menus remain public. Distinct from mennyuOrdersPaused.
  orderingEnabled Boolean @default(true)
}

model Vendor {
  /// Durable product mode. When false, this vendor is menu-only: public
  menus remain
  /// browsable but customers cannot add items to cart. Distinct from
  mennyuOrdersPaused.
  orderingEnabled Boolean @default(true)
}
```

Optional later (not required for v1): `Vendor.orderingEnabledChangedAt` , audit log actions `VENDOR_ORDERING_MODE_SET` / `POD_ORDERING_MODE_SET` .

Migration default: `true` for all existing rows. Production vendors must not become menu-only by surprise.

Indexes: not required for correctness. Add `@@index([orderingEnabled])` only if admin list filters need it.

3.2 Concepts that must stay separate

Concept	Source of truth	Menu-only meaning
Vendor exists	<code>Vendor</code> row, <code>deletedAt</code> null	Unchanged
Profile visible	<code>isActive</code> + public profile ready + <code>PodVendor.isActive</code> + <code>pod isActive</code>	Still visible
Menu published	<code>MenuVersion</code> published / operational catalog	Still published
Item available	<code>MenuItem.isAvailable</code>	Still meaningful for browsing ("sold out" vs in stock)
Currently open	Hours / <code>posOpen</code>	Still shown: Open · Menu only
Currently paused	<code>mennyuOrdersPaused</code>	Only applies when ordering is enabled
Allowed to accept OO orders	<code>orderingEnabled</code> (vendor) AND pod <code>orderingEnabled</code>	The new intent layer
Routing mode	<code>orderRoutingMode</code>	Preserved, unused for intake when menu-only
POS/integration readiness	connection + mappings	Irrelevant to public browse; required only when intent is on
Stripe payout readiness	Connect flags	Irrelevant to public browse; required only when intent is on

3.3 Derived resolver (centralize here)

Add a small module, conceptually `src/lib/vendor-ordering-mode.ts` , and feed it into the existing SSOT (`vendor-readiness-states.ts` , `vendor-orderability-in-`

pod.ts). Do not scatter `if (!vendor.orderingEnabled)` across UI files.

Recommended shape:

```
type VendorCommerceState = {
  podOrderingEnabled: boolean;
  vendorOrderingEnabled: boolean;
  effectiveOrderingEnabled: boolean; // pod AND vendor flags
  visibility: "hidden" | "visible";
  customerCanOrder: boolean;
  blockedReason:
    | "pod_ordering_disabled"
    | "vendor_ordering_disabled"
    | "pod_orders_paused"
    | "vendor_paused"
    | "vendor_closed"
    | "ordering_setup_incomplete" // Stripe/routing/mapping while intent is
on
    | "item_unavailable"
    | null;
  customerStatusLabel: string; // e.g. "Open • Menu only"
  customerBannerLine: string | null;
  adminEffectiveLabel: string; // e.g. "Menu-only (pod disabled)"
};
```

Rules:

1. If public profile is incomplete → hidden. Unchanged.
2. If `!pod.orderingEnabled` or `!vendor.orderingEnabled` → visible (if profile ready), **cannot order**, Stripe/POS are **not** blockers and **not** "setup required."
3. If both flags are true → existing operational checks apply (Stripe, routing, hours, pause).
4. Pause/hours never flip `orderingEnabled` .
5. Disabling ordering never writes `isAvailable` , `menuSource` , `orderRoutingMode` , or Stripe fields.

3.4 State table (customer behavior)

Assume the vendor is publicly visible (profile + published menu).

Pod orderingEnabled	Vendor orderingEnabled	Stripe ready	Routing ready	Hours	Pause	Customer sees

true	true	yes	yes	open	no	Open / Order now
true	true	yes	yes	closed	no	Closed
true	true	yes	yes	open	yes	Not accepting orders (paused)
true	true	no	yes	open	no	Visible; "Ordering setup incomplete" internally; not "Menu only"
true	true	yes	no	open	no	Same, routing incomplete
true	false	anything	anything	open	n/a	Open · Menu only
true	false	anything	anything	closed	n/a	Closed · Menu only
false	true (preserved)	anything	anything	open	n/a	Open · Menu only (pod blocked)
false	false	anything	anything	open	n/a	Open · Menu only
true	false	yes (leftover Connect)	yes (leftover Square)	open	no	Menu only; Stripe/Square idle

```
effectiveOrderingEnabled = pod.orderingEnabled &&
vendor.orderingEnabled .
```

Admin should see the distinction:

- Vendor explicitly menu-only (`vendor.orderingEnabled = false`)
- Vendor orderable but blocked by pod (`vendor.orderingEnabled = true` , `pod.orderingEnabled = false`)
- Intent on but not ready (`ordering_setup_incomplete`)

3.5 Enabling ordering: safer UX

Do not block the admin toggle on Stripe/routing. Admin may set `orderingEnabled = true` as intent. Customers still cannot order until payment + routing are ready. Dashboard shows "Ordering setup incomplete," not "Menu only" and not "Live."

Do **not** auto-create Stripe accounts or start Connect onboarding merely because the flag flipped. Show the existing payout/routing checklist only after intent is on.

Do **not** disable ordering by flipping every item to `isAvailable = false`.

3.6 Naming

Use `orderingEnabled` (boolean) on both models.

Avoid:

- `isOrderable` as a stored column (sounds derived)
- Overloading `orderRoutingMode`
- A single `ready` boolean
- Reusing `mennyuOrdersPaused`

Customer copy: **"Menu only"** as the primary chip. Use "Ordering unavailable" only for pod-wide disable helper text, not as the vendor's main status when the vendor is open.

4. Files / Components Requiring Changes

Severity key:

- **Critical** — wrong behavior would hide menus, take illegal orders, or destroy data
- **Required** — needed for a coherent menu-only product
- **Nice-to-have** — polish, filters, analytics, claiming

4.1 Prisma / schema

Path	Current responsibility	Problem	Proposed change	Severity
<code>prisma/schema.prisma</code> (<code>Pod</code> , <code>Vendor</code>)	Pod/vendor identity, pause, Stripe, routing, menu source	No durable orderability intent	Add <code>orderingEnabled Boolean @default(true)</code> on both; comments that this is distinct from <code>mennyuOrdersPaused</code>	Critical

New Prisma migration	—	—	Default true ; no backfill to false	Critical
<code>src/lib/admin-audit-log.ts</code>	Admin reason + action names	Pause actions exist; no mode-set actions	Add <code>POD_ORDERING_MODE_SET</code> / <code>VENDOR_ORDERING_MODE_SET</code>	Required

4.2 Shared types / helpers (do this first)

Path	Current responsibility	Problem	Proposed change
New <code>src/lib/vendor-ordering-mode.ts</code>	—	Logic would otherwise scatter	Intent + effective flag blocked reasons + customer/admin labels
<code>src/lib/vendor-readiness-states.ts</code>	Public vs operational SSOT	Operational missing always includes Stripe/POS; customer labels are only open / not accepting / closed	If <code>!effectiveOrdering</code> skip Stripe/POS/square/del as health blockers; add only labels; keep <code>customerCanOrder =</code>
<code>src/lib/vendor-orderability-in-pod.ts</code>	Cart/checkout gate	No <code>pod_ordering_disabled</code> / <code>vendor_ordering_disabled</code>	New reasons + copy: "vendor is menu-only" vs closed
<code>src/lib/vendor-availability.ts</code>	Open / closed / paused / inactive	Comment says <code>isActive</code> means "visible for ordering"; <code>orderable</code> means can receive orders	Keep hours/pause here put product mode here
<code>src/lib/vendor-pod-readiness.ts</code>	Checklists and <code>canAcceptOrders</code>	Stripe/POS always required; pod needs ≥ 1 orderable vendor	Split menu-ready vs or ready checklists; pod <code>vendor_ready</code> become publicly listed vendor" menu-only pods
<code>src/lib/vendor-readiness-validation.server.ts</code>	Loads Stripe/POS/menu bundles	Select list lacks <code>orderingEnabled</code>	Select new fields; pass evaluation
<code>src/lib/vendor-operational-copy.ts</code>	"Accepting orders" / "Not ready"	No menu-only label	Add "Menu only"; keep pause/closed distinct

<code>src/lib/pod-page-status.ts</code>	Pod ticker from vendor unavailable	All-menu-only pod → "Not accepting orders"	Menu-only pod → "Menu unavailable" / "Open for browsing"; mixed → "No vendors taking orders"
<code>src/lib/vendor-orderability-in-pod.test.ts</code>	Pause/inactive tests	No mode tests	New cases for flags vs Stripe
<code>src/lib/vendor-readiness-states.test.ts</code>	Public vs operational	Menu-only would currently be <code>needs_payment</code>	Assert Stripe not required when ordering disabled
<code>src/lib/vendor-pod-readiness.test.ts</code>	Checklist completeness	—	Menu-only vendor can setup-complete without

4.3 Admin

Path	Current responsibility
<code>src/app/admin/(dashboard)/vendors/[vendorId]/AdminVendorOverview.tsx</code>	Vendor rescue UI
<code>src/lib/admin-vendor-summary.ts</code>	Overall status, attention items
<code>src/services/admin-vendor-detail.service.ts</code>	Loads vendor detail
<code>src/services/admin-vendor-rescue.service.ts</code>	Pause/hide/routing/menu source repair
<code>src/actions/admin-vendor.actions.ts</code>	Platform-admin mutations
<code>src/app/admin/(dashboard)/pods/[podId]/AdminPodOverview.tsx</code>	Pod rescue UI

<code>src/lib/admin-pod-summary.ts</code>	<code>no_orderable_vendor</code> overall status
<code>src/services/admin-pod-detail.service.ts</code> / <code>admin-pod-rescue.service.ts</code> / <code>admin-pod.actions.ts</code>	Pod pause/hide/attach
<code>src/app/admin/(dashboard)/vendors/page.tsx</code>	Search + routing filter
<code>src/services/admin-vendor-detail.service.ts</code> <code>searchAdminVendors</code>	Routing filter only
<code>src/app/admin/(dashboard)/pods/page.tsx</code> + <code>AdminPodsTable.tsx</code>	Search; readiness column
<code>src/app/admin/(dashboard)/vendors/[vendorId]/page.tsx</code>	Composes overview

Admin Vendor Detail — visibility when menu-only

Always visible: identity, public profile, pods, menu publish status, hours, owners/claim status, visibility (hide/show), **ordering mode**, pause (secondary), audit log, public URL.

Hide or collapse to Advanced when menu-only: payout/Stripe setup, routing editor, POS/Square/Deliverect warnings, tablet presence, kitchen, order-issue panels, GMV/order metrics. Do not delete the data. If leftover Stripe exists, show a quiet "Payouts connected (idle)" line in Advanced.

Admin Pod Detail — control hierarchy

1. Pod-level **Ordering enabled** is the master customer switch.
2. Per-vendor **Menu only / Orderable** is intent, preserved when the pod is off.
3. Pause remains a temporary override under Ordering controls.
4. Do not duplicate the pod toggle on every vendor row. Show a badge: `Menu-only` (pod) vs `Menu-only` (vendor) .

4.4 Customer storefront

Path	Current responsibility	Problem
<code>src/lib/vendor-menu-customer-page-render.tsx</code>	Vendor menu page	Maps any non orderable to

		paused/inactive creates a cart; orderingDisabled fed into item card
src/components/vendor-menu/VendorMenuItemCard.tsx	Item card	itemUnavailable orderingDisabled !item.isAvailable
src/components/vendor-menu/useMenuItemAddAction.ts	Add / customize	Respects orderingDisabled but used as unavailable
src/app/pod/[podId]/vendor/[vendorId]/AddToCartButton.tsx	Qty / add CTA	Hides/disables orderingDisabled
src/app/pod/[podId]/vendor/[vendorId]/ModifierModal.tsx	Customize + add	No orderingDisabled of its own
src/components/vendor-menu/VendorMenuHero.tsx	Open / Closed / Not accepting / Unavailable	No menu-only
src/lib/pod-customer-page-data.ts	Pod vendor grid	unavailable orderable
src/components/pod/PodVendorCard.tsx	CTA "Order now" / "View menu"	"View menu" card when unavailable

<code>src/lib/pod-page-status.ts</code>	Pod-level ticker	See above
<code>src/components/pod/PodPageGroupOrderSection.tsx</code> + <code>src/lib/pod-page-group-order-cta.ts</code>	Start group order	Assumes order
<code>src/app/explore/page.tsx</code> + <code>src/lib/explore-discovery.ts</code>	Discover pods/vendors	Filters <code>isActive</code> only; "Open no from pause, no readiness
<code>src/lib/vendor-menu-spotlight.ts</code>	Featured/popular sections	Fine
QR / deep links (<code>src/app/admin/.../qr</code> , promote pages)	Public URLs	Fine if pod <code>status.isActive</code>

Deep links and QR keep working because public routes key off `isActive` + public profile, not Stripe.

4.5 Cart

Path	Current responsibility	Problem	Proposed change
<code>src/services/cart.service.ts</code> <code>addCartItem</code>	Server add	Will work once orderability helper knows the flags	New error codes <code>VENDOR_ORDERING_DISABLED</code> / <code>POD_ORDERING_DISABLED</code> ; message distinct from pause
<code>src/services/cart.service.ts</code> <code>updateCartItem</code>	Qty / modifiers	Does not re-check vendor orderability	Re-run <code>getVendorOrderabilityInPod</code> with readiness
<code>src/services/order.service.ts</code> <code>validateCartForOrder</code>	Checkout validation	Will pick up helper	Assert new codes in tests
<code>src/services/order.service.ts</code> <code>validateCartItemsForDisplay</code>	Cart page errors	Skips Stripe/POS; would skip	Use full readiness or at least the new flags + pause

		new flags unless added to shallow path	
<code>src/lib/cart-for-validation.ts</code>	Cart → validation DTO	May omit new fields	Include <code>orderingEnabled</code> for pod + vendor
<code>src/app/cart/page.tsx</code> + cart item UI	Display errors	Generic "not accepting"	Per-vendor: menu-only vs paused vs closed vs sold out; allow removing invalid lines; keep other vendors
<code>src/actions/cart.actions.ts</code> / <code>src/app/api/cart/route.ts</code>	Mutations	Fine if service layer is correct	No bypass

Recommended cart behavior when a vendor/pod flips to menu-only:

- Cannot add new items (API error).
- Existing lines from that vendor fail validation with a clear reason.
- Customer can remove those lines; other orderable vendors remain.
- Do not auto-delete silently; show why.
- Do not treat as "item sold out" or "vendor closed."

4.6 Checkout / payments

Path	Current responsibility	Problem	Proposed change
<code>src/services/order.service.ts</code> <code>createOrderFromCart</code>	Creates Order + VendorOrders	Uses <code>validateCartForOrder</code>	Sufficient if helper is wired; add tests for menu-only
<code>src/app/checkout/page.tsx</code>	SSR validation	Uses display validator	Must see new errors before <code>PaymentIntent</code>
<code>src/app/api/checkout</code>	POST checkout	—	Keep server reject
<code>src/services/payment.service.ts</code> <code>createPaymentIntent</code>	PI for existing pending order	No vendor re-check	Acceptable if order create is the gate;

			optional belt-and-suspenders reject
Stripe Connect / allocation / commission	Per paid order	Menu-only vendors should never enter checkout	No change if gated earlier
Pod with zero orderable vendors	—	Should not attempt payment or show Connect errors on public pages	Public browse has no checkout CTA

Menu-only vendors must never be forced through payout recipient validation just because they sit on the pod.

4.7 Vendor dashboard

Current nav (`VendorAreaNav`): Dashboard, Orders, Menu, Hours, Payouts, Setup, Vendor Profile, Kitchen. Also pages: issues, analytics, connect-pos, integrations/square, menu-imports.

Path	Current responsibility	Problem	Prop
<code>src/app/vendor/[vendorId]/VendorAreaNav.tsx</code>	Always shows Orders/Payouts/Kitchen	Irrelevant ops for menu-only	Con (bel
<code>src/app/vendor/[vendorId]/VendorLayoutChrome.tsx</code>	Layout	Passes only routing mode	Pass ord / eff
<code>src/app/vendor/[vendorId]/layout.tsx</code>	Auth + chrome	Does not load ordering flags	Sele
<code>src/app/vendor/[vendorId]/dashboard/page.tsx</code>	Live orders, pause, performance	Assumes ordering	Men dash prof statu ticke
<code>src/lib/vendor-dashboard-data.server.ts</code>	<code>canAcceptOrders</code> , <code>setupComplete</code>	Setup complete includes Stripe	Men com prof hou

src/lib/vendor-dashboard-attention.ts	Stripe/POS banners	Always nag Connect	Support ordering nags only
src/app/vendor/[vendorId]/setup/page.tsx	Full ordering checklist	—	Menu checker
src/app/vendor/[vendorId]/orders/page.tsx	Order history + kitchen CTA	—	Hide history order history
src/app/vendor/[vendorId]/kitchen/page.tsx	Kitchen mode	—	Hide vendor
src/app/vendor/[vendorId]/payouts/page.tsx	Stripe Connect	—	Hide Advanced if already connected
src/app/vendor/[vendorId]/connect-pos/page.tsx + integrations/**	Routing	—	Hide enable
src/app/vendor/[vendorId]/issues/page.tsx	Order issues	—	Hide exist
src/app/vendor/[vendorId]/hours/page.tsx	"Customer ordering hours"	Copy assumes ordering	"Hours menu same"
src/app/vendor/[vendorId]/menu-builder/page.tsx	Menu builder	Does not require ordering	Keep
src/app/vendor/[vendorId]/analytics/page.tsx	Order analytics	Zero GMV looks like failure	Hide metrics empty engagement
src/app/vendor/[vendorId]/vendor-dashboard-redesign.test.ts	Asserts full nav	Will fail if nav changes	Update

Recommended menu-only vendor nav

- Dashboard
- Menu
- Hours

- Vendor Profile
- Setup (menu/profile only)

If the vendor has **open VendorOrder** s when switched to menu-only: keep **Orders** (and Kitchen if manual routing) until those tickets finish. Historical orders: add **Order history** rather than the live ops home.

When ordering is re-enabled, restore Orders / Kitchen / Payouts / routing without recreating the account.

Do not hide individual cards and leave empty Orders/Payouts landing pages in the nav.

4.8 Pod owner dashboard

Path	Current responsibility	Pro
src/app/pod/[podId]/dashboard/PodDashboardSidebar.tsx	"N orderable vendors"	Co cor
src/app/pod/[podId]/dashboard/PodDashboardSetupChecklist.tsx	"find and order"	—
src/lib/pod-dashboard-attention.ts	no_orderable_vendors warning	Fal unl
src/lib/pod-dashboard-data.server.ts	Metrics + roster	GM alw
src/app/pod/[podId]/dashboard/PodDashboardMetrics.tsx	Orders today / GMV	Zer de: loc
src/app/pod/[podId]/dashboard/PodVendorReadinessSection.tsx + roster/adoption	"Needs Stripe"	Me flag ado fail

<code>src/lib/pod-vendor-adoption.ts</code>	<code>needs_payment</code> → "Needs Stripe"	—
<code>src/app/pod/[podId]/vendors/PodVendorsPageView.tsx</code>	Filters live / not accepting	—
<code>src/app/pod/[podId]/setup/PodReadinessVendorSection.tsx</code>	All vendors should <code>canAcceptOrders</code>	—
Promote / QR / branding / hours / events	Remain	"Sc orc
Pod payouts	Optional	Fin kee set do po me

Mixed pod metrics: show both. Commerce cards apply only to orderable vendors. Do not average menu-only vendors into "0% orderable = failing."

Do not grant pod owners the platform orderability toggle in v1 unless you explicitly want it. Pause-in-pod (`PodVendor.isActive`) already exists for owners and **hides** the vendor — the opposite of menu-only.

4.9 Routing / integrations

Path	Current responsibility	Problem	Proposed change
<code>src/lib/vendor-order-routing-mode.ts</code>	Routing readiness	Used as operational requirement	Only when <code>effectiveOrderingEnal</code>
<code>src/lib/vendor-menu-source.ts</code>	Menu source from routing	Must not run on orderability toggle	No writes from ordering mode changes
<code>src/services/vendor-menu-source-ownership.service.ts</code>	Ownership repair	Easy to call from rescue	Ordering-mode service n not import/reconcile this
Square / Deliverect adapters	Inject paid orders	Fine if no new orders	Leave config in place
<code>src/services/deliverect-busy-mode-webhook.service.ts</code>	Writes <code>mennyuOrdersPaused</code>	Must never write <code>orderingEnabled</code>	Document + test

Menu-only vendors do **not** need a routing mode to display a menu. Saved Square/Deliverect config stays idle. Enabling ordering later uses existing config; if missing, then show routing setup.

4.10 Menu

Path	Current responsibility	Problem	Proposed change	Severity
Menu Builder actions/data	Categories, items, modifiers, publish	Independent of ordering already	Keep available; publishing must not require Stripe	Critical
<code>src/lib/vendor-menu-management.ts</code>	Builder vs import path	Tied to routing mode, not orderability	Unchanged	Required
<code>MenuItem.isAvailable</code>	Sold out	Must not be used as vendor orderability	Explicitly forbidden in implementation prompts	Critical
Import/publish pipelines	Deliverect/Square/OO	Unchanged	Do not unpublish on menu-only	Required

4.11 Onboarding / readiness / Stripe

Path	Current responsibility	Problem	Proposed change
<code>src/lib/vendor-pod-readiness.ts</code>	Collapsed "ready"	Stripe+POS in required keys	Menu readiness vs Ordering readiness (section 10)
<code>src/services/stripe-connect.service.ts</code>	Connect onboarding	Triggered from payouts UI	Only surface when ordering enabled (or leftover account)

<code>src/actions/vendor-stripe-connect.actions.ts</code>	Vendor starts Connect	—	Guard: allow if ordering enabled or account already exists
<code>src/components/vendor/VendorSetupStatusBanners.tsx</code>	Incomplete setup	—	Menu-only complete without payouts
<code>AccountOnboardingStatus</code>	Lightweight UI enum	Not used as launch gate	Leave; do not overload

4.12 Notifications

Path	Current responsibility	Problem	Proposed change	Severity
<code>src/services/sms.service.ts</code>	OTP + order milestones	Order SMS only	No menu-only SMS needed; do not add payout nags	Require
<code>src/lib/pod-vendor-adoption.ts</code> <code>buildVendorAdoptionReminderMessage</code>	Stripe/hours reminders	Would nag menu-only vendors	Skip payment/routing reminders when <code>!orderingEnabled</code>	Require
Email verification / invites	Account	Fine	Unchanged	—

There is no existing Stripe-onboarding email campaign. Don't add one for menu-only vendors.

4.13 Analytics

Path	Current responsibility	Problem	Proposed change
------	------------------------	---------	-----------------

<code>src/services/pod-analytics.service.ts</code>	Orders, GMV, routing health	Zero GMV looks like a dead pod	Split engagement vs commerce (Phase I)
<code>src/services/vendor-analytics.service.ts</code>	Vendor GMV/orders	Same	Same
<code>src/app/admin/(dashboard)/analytics/page.tsx</code>	Platform commerce	—	Don't treat menu-only pods as failed
<code>src/components/retention/RecentViewTracker.tsx</code>	Client recent views	Local, not warehouse	Closest existing "view" signal; not a full analytics system

Today there are no first-class pageview / QR-scan / item-view events. Ordering metrics will be zero by design. Do not build a warehouse in the first slice; do stop calling zero GMV "needs attention."

4.14 Authorization

Path	Current responsibility	Problem	Proposed change	Severity
<code>src/lib/admin-action-context.ts</code> + <code>requireAdminActionContext</code>	Platform admin mutations	Correct owner for v1 toggles	New set-mode actions use this only	Critical
<code>src/lib/permissions.ts</code>	<code>isPlatformAdmin</code>	Pod owners should not get v1 toggle	Do not add to pod dashboard actions	Required
Cart/checkout APIs	Session/group actor	Stale clients	Server reject is the real control	Critical

4.15 Existing orders

Vendor order queries are by `vendorId` / order id, not by pause or Stripe. Historical and in-flight orders should keep showing.

Path	Change	Severity
Vendor orders board / kitchen / admin order detail	Do not filter out because <code>orderingEnabled = false</code>	Critical
Routing of new orders	Blocked by checkout	Critical
Routing of existing orders	Continue	Critical

5. Hidden Couplings / Risks

1. `orderingDisabled === item unavailable ( VendorMenuItemCard )`. Highest UX risk. Menu-only would look sold out.
2. **Menu-source ownership.** `adminPauseVendorOrdering` already revalidates menu caches (safe). A new mode setter must **not** call `reconcileVendorMenuSourceOwnership` or `adminUpdateVendorOrderRoutingMode`. Do not “helpfully” switch routing to `manual_dashboard` when disabling ordering.
3. **Deliverect busy mode** → `mennyuOrdersPaused`. If someone reuses pause for menu-only, POS webhooks will fight the product flag.
4. `getVendorOrderabilityInPod` without **readiness fail-opens**.
Display/update paths that skip readiness would also skip the new flags unless the flags are checked in the shallow branch too. Put flag checks in **both** branches.
5. **Public vendor page hardcodes pause false during visibility check** (`vendor-menu-customer-page-render.tsx` lines 54–55), then uses real pod pause for orderability. Keep that split; add `orderingEnabled` to the orderability evaluation, not the visibility short-circuit.
6. **Pod isActive = false 404s the whole pod**. Menu-only pods must stay active. Do not use hide-pod as menu-only.
7. **PodVendor.isActive = false hides the vendor**. Owners already have a hide control. Do not use it as menu-only.
8. **Hours field name customerOrderingHours**. Keep the column; change labels for menu-only. Requiring hours for public appearance is still correct (customers should see when the stall is open).
9. **Group orders** on destination/standard pod pages. A menu-only pod must not offer “Start group order.”
10. **Cart still created on menu pages** (`getOrCreateCartForVendorMenuAction`).
Harmless if add is blocked; mixed pods still need a cart for orderable vendors.

11. **Explore “Open now”** is shallow. Menu-only open vendors should not be advertised as orderable.
 12. **Admin “Open” vendor filter** = `accepting_orders` . Menu-only vendors would vanish from “Open” if that filter stays commerce-only — rename to “Accepting orders” and add “Menu only.”
 13. **Existing in-flight work** (menu-source ownership / availability repair in the working tree) must not be mixed into this feature. Orderability is a different axis.
 14. **Stripe leftover on menu-only.** Do not disconnect Connect. Do not require it. Do not show “payouts not ready” as a launch blocker.
 15. **Tests that assert full vendor nav** (`vendor-dashboard-redesign.test.ts`) will fail when nav is conditional — update them in Phase E, not by keeping ghost links.
-

6. Recommended Implementation Sequence

Phase A — State model

Schema + `vendor-ordering-mode.ts` + thread into `vendor-readiness-states` and `vendor-orderability-in-pod` . Defaults `true` . Unit tests for the state table. **No UI yet except tests.**

Phase B — Admin controls

Vendor Detail + Pod Detail toggles, distinct from pause. Audit log. Vendor row effective badges. Platform admin only. No menu-source writes.

Phase C — Storefront

Menu-only browsing: prices, hours, sold-out, **no add-to-cart**, no customize modal, “Menu only” chip, pod ticker, vendor card CTA, group-order CTA hidden when nobody is orderable.

Phase D — Cart / checkout enforcement

New codes on `addCartItem` , `updateCartItem` , `validateCartForOrder` , `validateCartItemsForDisplay` . Distinct messages vs pause/closed/unavailable. Multi-vendor carts keep valid vendors.

Phase E — Vendor dashboard

Conditional nav and dashboard home. Keep Orders/Kitchen if open tickets exist. Menu Builder always on.

Phase F — Pod dashboard

Attention/metrics/roster copy. Mixed-pod metrics. No owner toggle in v1.

Phase G — Readiness / payments / routing cleanup

Split checklists. Suppress Stripe/POS nags when menu-only. Enabling ordering reveals checklist; does not auto-Connect.

Phase H — Vendor claiming / concierge onboarding

Admin create vendor **without** a user. Public unclaimed listing (already possible if the row exists).

Claim invite for an existing vendor (new token type or extend `PodVendorInvite`). Not required to ship menu-only browse.

Phase I — Analytics and polish

Engagement vs commerce. Admin list filters. Copy sweep. Explore badges.

7. Testing Plan

Unit

- State table in §3.4 (every row).
- Pause vs menu-only vs closed vs sold-out vs hidden vs `ordering_setup_incomplete` .
- Pod flag off preserves vendor `orderingEnabled = true` and still blocks cart.
- `getVendorOperationalMissingItems` does not include `stripe / pos` when ordering disabled.
- Menu-source ownership functions are **not** called from the new setter (spy/test).
- Deliverect busy-mode still only writes `mennyuOrdersPaused` .
- `VendorMenuItemCard` : menu-only + `isAvailable true` is not greyed-out sold-out; menu-only + `isAvailable false` still sold-out.
- Pod ticker for 0/10/2-of-10 orderable vendors.
- Cart line messages: new codes $\neq$ `VENDOR_PAUSED_MENNYU` $\neq$ `VENDOR_CLOSED` $\neq$ `ITEM_UNAVAILABLE` .

Integration

- `addCartItem` rejects menu-only vendor and pod-disabled pod.
- `updateCartItem` rejects after flag flip.
- `createOrderFromCart` rejects even if cart already contains lines.
- Mixed cart: menu-only vendor lines fail; orderable vendor lines can still check out after removal.

- Existing `VendorOrder` still loads on vendor orders/kitchen/admin after menu-only.
- Publish menu as menu-only vendor without Stripe.
- Public pod + vendor pages 200 with `orderingEnabled false` and no Stripe.
- Public page 404 still when public profile incomplete.
- Admin non-admin cannot call set-mode actions.
- Re-enable ordering: Stripe-missing vendor is `visible_not_accepting`, not live; Stripe-ready vendor can add to cart.

Migration

- Existing rows `orderingEnabled = true`.
- App boot with mixed flags.
- Idempotent migrate deploy on staging.

Manual QA

1. Entire pod menu-only: browse all vendors, no cart CTAs, QR works, pod dashboard not in "crisis" mode.
2. 8 menu-only + 2 orderable: order only from the two; cards differ; mixed ticker.
3. Disable pod ordering while vendors stay `orderingEnabled true`; re-enable; those two accept orders again without republishing menus.
4. Flip one vendor to menu-only with items in a multi-vendor cart; other vendor remains.
5. Active ticket in kitchen, then set menu-only; ticket still completable.
6. Menu-only vendor with leftover Square/Stripe: no disconnect; Advanced shows idle.
7. Enable ordering on a vendor with no Stripe: admin toggle succeeds; customer cannot add; checklist appears.
8. Hours open/closed still display on menu-only.
9. Sold-out item on menu-only still sold-out, not "unavailable vendor."
10. Regression: fully orderable tablet vendor can still order end-to-end.

Regression protection

- Do not break current orderable vendors (default true).
- Menu-source ownership tests must stay green and unused by this feature.
- `validateCartForOrder` existing pause/hours/Stripe tests remain.

8. Recommended First Implementation Slice

Ship the smallest slice that yields:

A real menu-only pod with menu-only vendors that can be publicly browsed, with add-to-cart blocked on client and server, without changing current orderable vendors.

In scope

1. `Pod.orderingEnabled` + `Vendor.orderingEnabled` default `true` .
2. Central resolver + orderability reasons.
3. Admin toggles on Vendor Detail and Pod Detail (platform admin), **separate from pause**.
4. Storefront: "Menu only", hours, prices, sold-out, **no add / no customize**.
5. `addCartItem` + `validateCartForOrder` + `validateCartItemsForDisplay` + `updateCartItem` reject with distinct codes.
6. Do not treat zero orderable vendors as a public-launch failure.
7. Do not require Stripe for public appearance (already true) and do not nag it when flags are off.

Out of scope for slice 1

- Vendor nav overhaul (Phase E) — acceptable to leave Orders/Payouts links as long as they are not required and do not block browse. Prefer a minimal nav hide if cheap.
- Pod-owner toggle
- Claim-unclaimed-vendor
- Analytics warehouse
- Disconnecting Stripe/Square
- Changing `menuSource` / routing

Success criteria

- Existing production-like vendor with Stripe + routing still orders.
- Admin sets pod or vendor to menu-only.
- Customer opens QR → pod → vendor menu → sees items and prices → cannot add to cart.
- Direct POST add-to-cart returns `VENDOR_ORDERING_DISABLED` or `POD_ORDERING_DISABLED` .
- Menu data, routing rows, and Stripe ids unchanged.
- Re-enable restores ordering without republishing the menu.

Slice 1 implementation prompt (for a later Cursor session)

Use this as the next focused prompt after review:

1. Add `orderingEnabled` Boolean `@default(true)` to `Pod` and `Vendor` ; migrate.
2. Implement `getVendorCommerceState` / extend `getVendorOrderabilityInPod` with `pod_ordering_disabled` and `vendor_ordering_disabled` . Never set `MenuItem.isAvailable` or call menu-source ownership.
3. Admin: `adminSetVendorOrderingEnabled` / `adminSetPodOrderingEnabled` via `requireAdminActionContext` . UI next to existing pause controls, labeled so pause ≠ menu-only.

4. Storefront: split `orderingDisabled` from item unavailability in `VendorMenuItemCard` and hero/card copy.
5. Cart/checkout: reject in add/update/validate/createOrder.
6. Tests for the state table and a mixed cart.

Do not start that work until this audit is reviewed.

Appendix A — Edge-case matrix

#	Scenario	Expected
1	Entire pod menu-only	Public pod OK; no order CTAs; dashboards not “failing”
2	10 menu-only vendors	All listed; none orderable
3	8 menu-only + 2 orderable	Mixed CTAs; cart only from the two
4	Pod ordering off, vendors still <code>orderingEnabled true</code>	Customer blocked; flags preserved; admin shows “blocked by pod”
5	Pod ordering re-enabled	Vendors with intent+readiness go live immediately
6	Vendor ordering off	That vendor menu-only; others unchanged
7	Vendor ordering re-enabled	Restore commerce if Stripe/routing ready; else setup-incomplete
8	Menu-only with Stripe already	Idle Connect; no disconnect; no payout nag
9	Menu-only with Square/Deliverect saved	Config preserved; no injection of new orders
10	Integrated → menu-only	Stop new orders; keep integration rows
11	Menu-only → tablet ordering	Set flag true; routing <code>manual_dashboard</code> already default; show kitchen if ready
12	Unclaimed but public	Allowed by schema; no create-unclaimed UI today
13	Vendor claims account	Phase H; membership attach; ordering flags unchanged
14	Cart has items, vendor becomes menu-only	Block those lines; explain; don’t call it sold out
15	Multi-vendor cart, one flips	Remove/block one vendor; keep the other
16	Active order, then menu-only	Fulfill normally; keep ticket UI
17	Historical orders	Remain in admin/vendor history

18	Pod owner, zero orderable	Overview/QR/vendors; no GMV crisis
19	Vendor dashboards	Unclaimed: no login; menu-only claimed: menu/profile; orderable: current ops
20	Published menu, ordering off	Browse all available items; sold-out still sold-out

Appendix B — Proposed readiness model

Menu readiness (public appearance)

- Vendor `isActive` , not deleted
- Attached to pod with `PodVendor.isActive`
- Pod `isActive`
- Name, description, banner, cuisine
- Published/operational menu
- Hours configured

Not required: Stripe, routing, POS, claimed owner, `orderingEnabled` .

Ordering readiness (paid intake)

- Menu ready
- `vendor.orderingEnabled`
- `pod.orderingEnabled`
- Stripe charges + payouts (when taking OO payments)
- Valid routing method + mapping if that method needs it
- Not paused
- Currently open
- At least one available item

Never collapse these into one `ready` boolean in APIs. Expose `menuReady` , `orderingIntent` , `orderingReady` , `customerCanOrder` .

Appendix C — Copy that must become conditional

Current copy	Where	Menu-only / mixed replacement
"Open for orders"	Hero, pod ticker, readiness	"Open" or "Open · Menu only"

"Not accepting orders right now"	Hero, cards, banners	Keep for pause ; do not use for menu-only
"Order now"	PodVendorCard	"View menu"
"Start group order"	Pod page	Hide if none orderable
"Ready for orders"	vendor-pod-readiness	"Ready to appear on the pod" vs "Ready to accept orders"
"At least one orderable vendor"	Pod checklist	"At least one published vendor" for menu-only pods
"Connect Stripe payouts"	Setup	Hide unless ordering enabled
"Set up payments, menu, and notifications"	Account tools	Payments only if ordering
"finish the checklist so customers can order"	Vendor dashboard	"so customers can find your menu"
"N orderable vendors"	Pod sidebar	Listed vs orderable counts
"No vendors are currently orderable"	Attention	Only if pod ordering is on
"Customer ordering hours"	Hours / setup	"Hours" when menu-only
"Scan to order" / QR	Promote	"View menus at this pod"
Kitchen / Payouts nav	Vendor chrome	Hide when menu-only (Phase E)

Appendix D — Minimum public menu-only pod

A pod is publicly usable for browsing when:

1. `Pod.isActive` and not deleted
2. Profile enough to render (name; image/description already in pod checklist)
3. ≥ 1 attached vendor that is **menu-ready** (public profile + published menu)
4. QR/slug works

Not required: Stripe, routing, POS, orderable vendors, claimed vendor users, pod payouts.

`Pod.orderingEnabled = false` is compatible with this. Vendor-level menu-only is also compatible while the pod remains orderable for other stalls.